



برنامه نویسی سیستمی

جلسه اول: معماری سیستم و آماده سازی محیط

تا امروز راننده بودید، حالا مکانیک می‌شوید

User Space

SPLIT

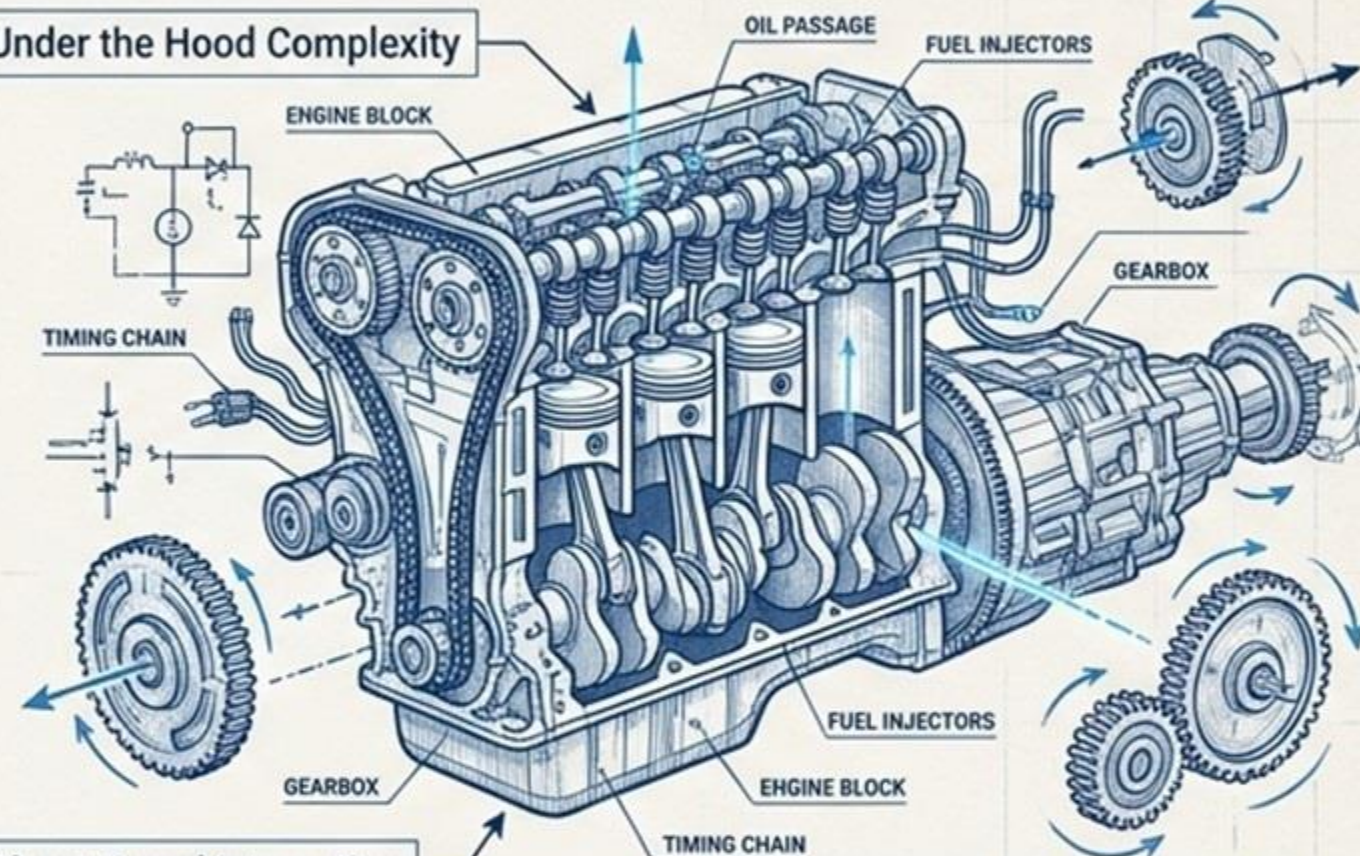
System Space



Comfortable Driver

Simplified Interface

Under the Hood Complexity



Direct Kernel Interaction

لایه User Space

تا پیش از این، درگیر اسکریپت‌های پایتون و طراحی وب بودید. مدیریت RAM و هارد دیسک از شما پنهان بود.

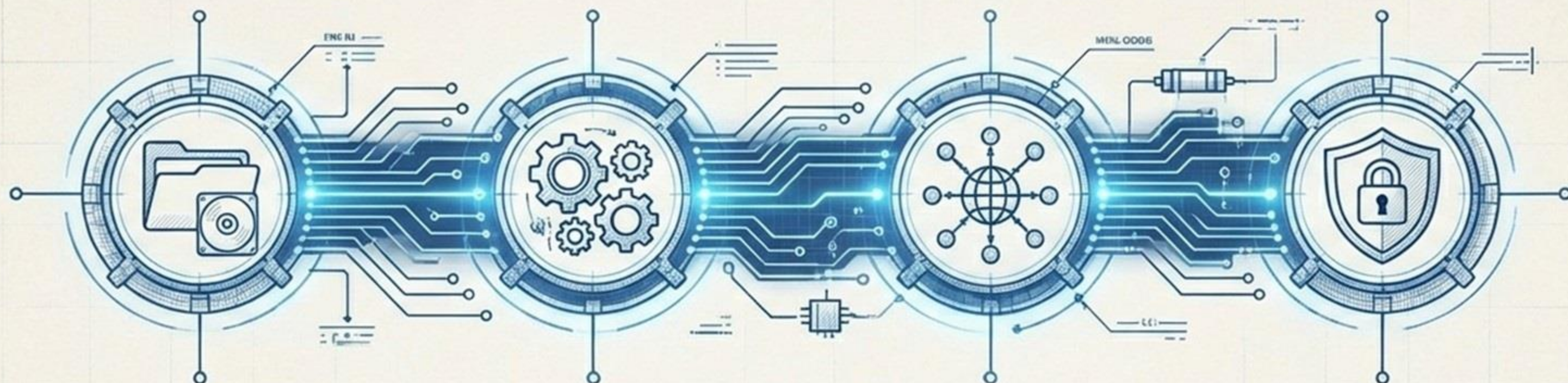
مفهوم System Programming

یعنی بالا زدن کاپوت ماشین. در این درس، ما مستقیماً با «موتور» (Kernel) صحبت می‌کنیم.

منابع جدید

تمرکز ما بر منابع پنهان است: مدیریت فایل‌های سطح پایین، پروسسرها، حافظه، و شبکه.

نقشه راه ترم



بخش ۱: معماری و فایل‌ها

جلسات ۱-۳: تعامل با لایه هسته، System Calls

بخش ۲: مدیریت پروسس

جلسات ۴-۶: مفاهیم Forking، Thread، و همگام‌سازی

بخش ۳: شبکه و سوکت

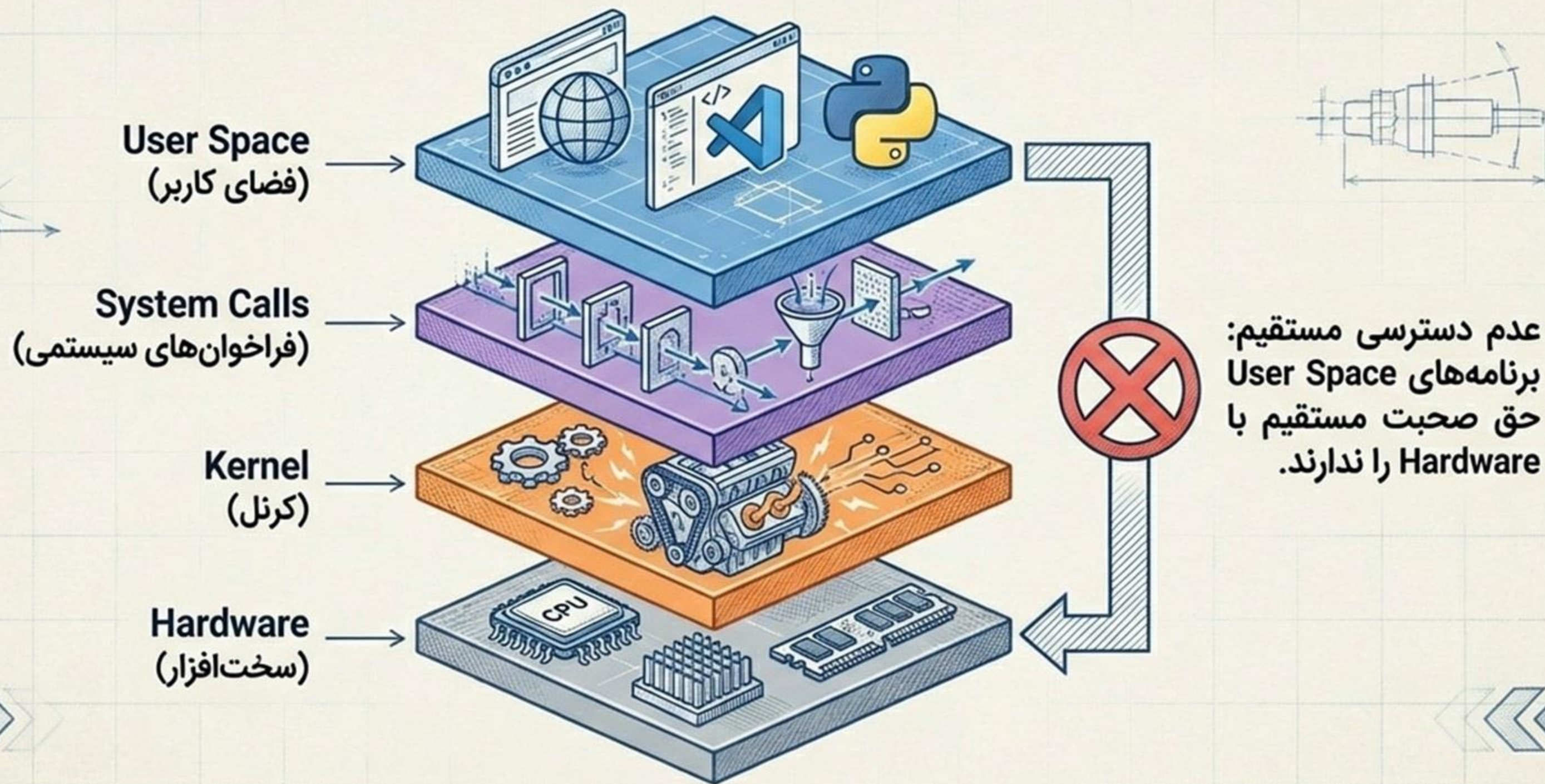
جلسات ۷-۹: ارتباط TCP/UDP و اسکنر پورت

بخش ۴: امنیت و پروژه

جلسات ۱۰-۱۲: تحلیل بدافزار و امنیت

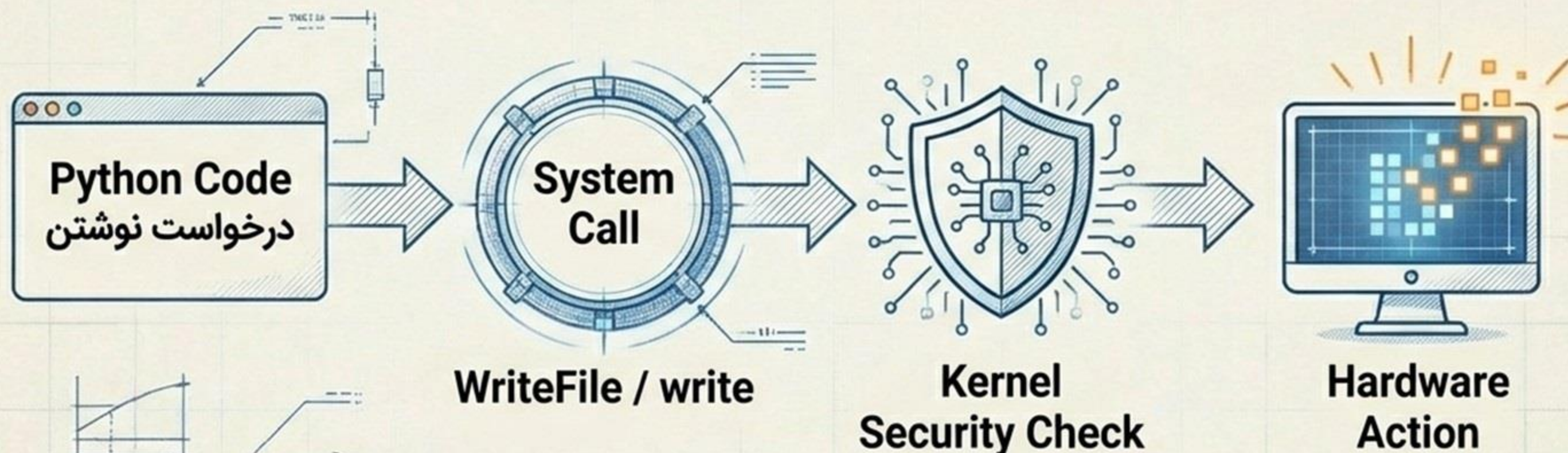
رویکرد ترکیبی: ویندوز + لینوکس | زبان: پایتون با مفاهیم C/C++

تصویر بزرگ: کامپیوتر چگونه کد شما را می‌فهمد؟



نگهبان دروازه: مفهوم System Call

یک برنامه معمولی نمی‌تواند خودسرانه روی هارددیسک بنویسد. باید «مؤدبانه» از کرنل درخواست کند. این درخواست مؤدبانه، System Call نام دارد.



تفاوت در عمل: کد سطح بالا در برابر کد سیستمی

High Level

```
python x  
pythn.py > ...  
1 print('Hello World')  
2
```

ساده، راحت، اما جزئیات پنهان است.

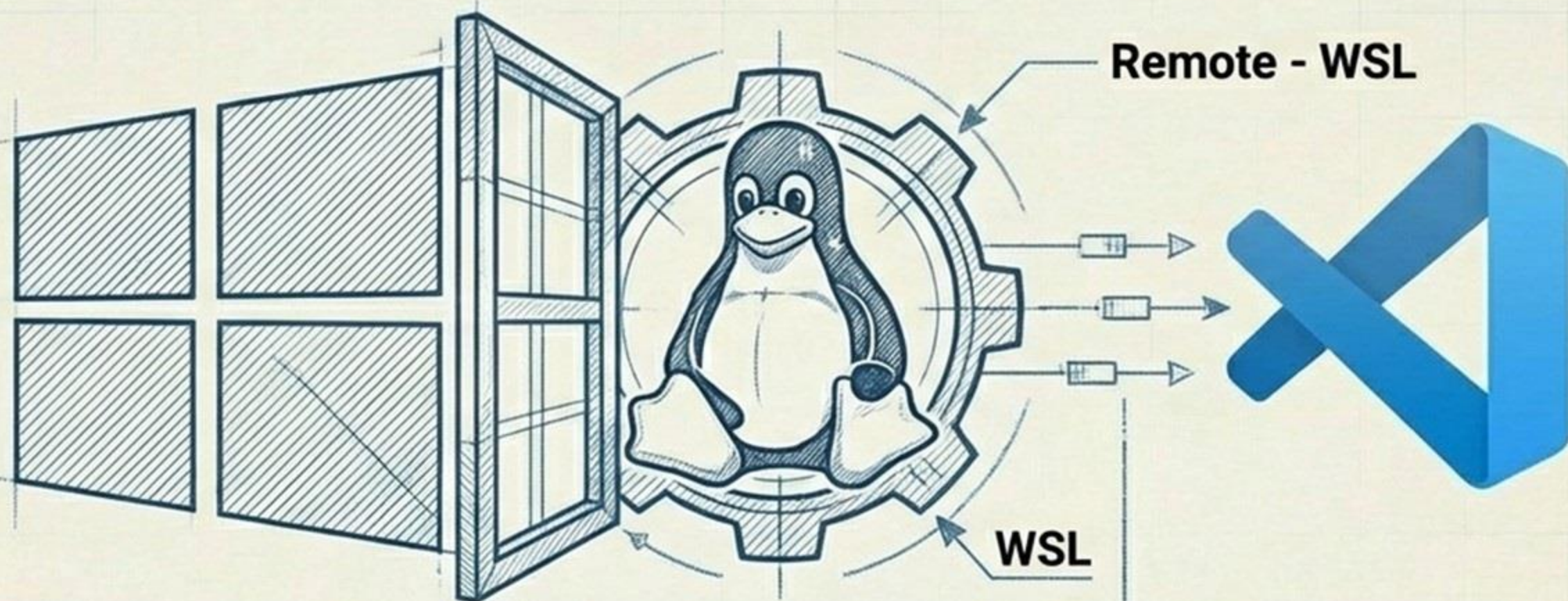
System Level

```
python x  
python os > ...  
1 import os  
2  
3 os.write(1, b'Hello System World\n')
```

File Descriptor 1 = stdout
(مانیتور)

مستقیم، خام، و کنترل شده.

جعبه ابزار مدرن: چرا WSL و VS Code؟



Why Linux?

لینوکس زبان مادری
مفاهیم سیستمی است.

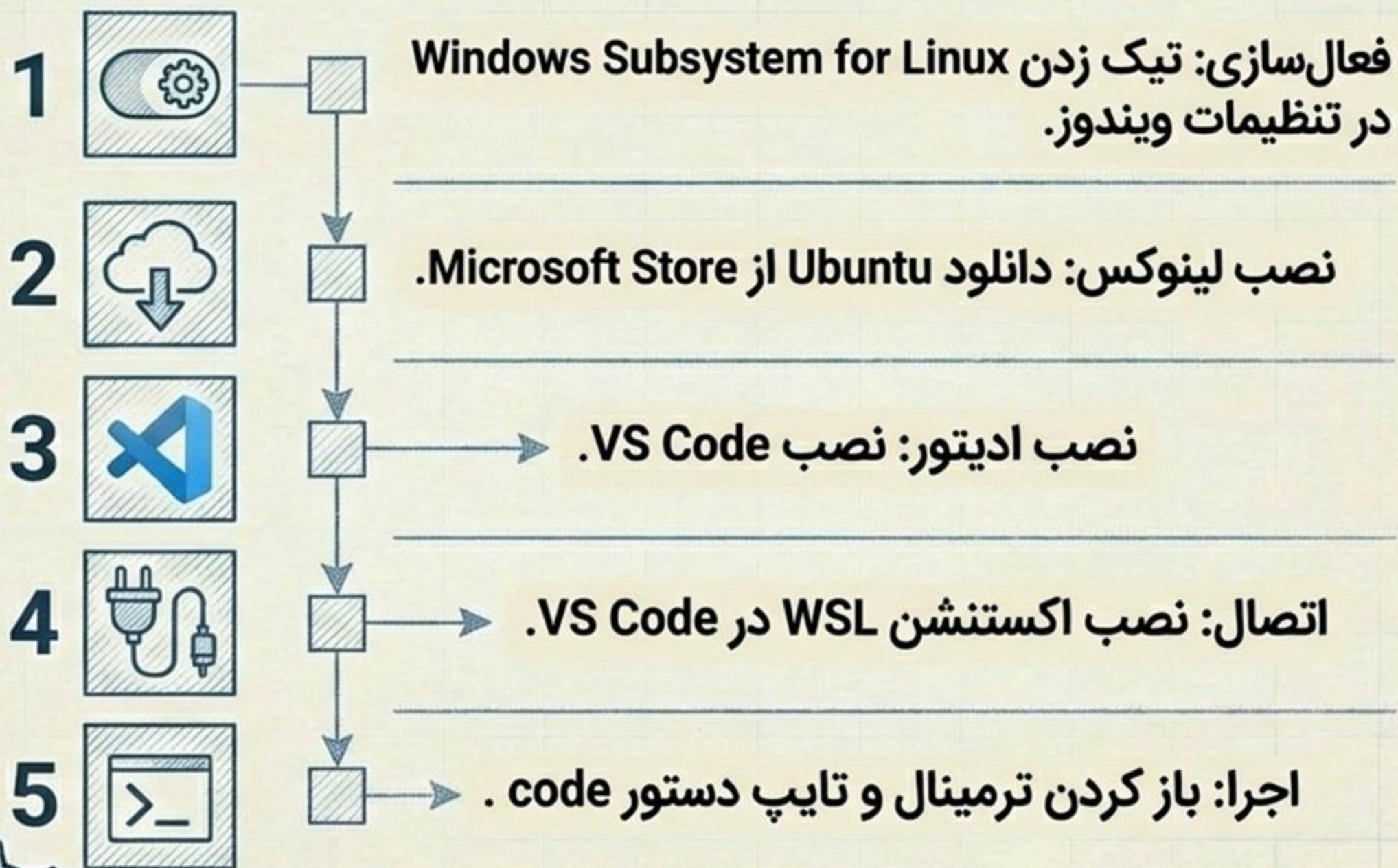
Why WSL?

اجرای لینوکس واقعی (Ubuntu)
در دل ویندوز، بدون نیاز به
ماشین مجازی (VM).

Why VS Code?

اکستنشن: Remote - WSL:
کدنویسی در ویندوز، اجرا در
لینوکس.

راهنمای سریع راه‌اندازی



اولین مأموریت: استخراج اطلاعات حیاتی سیستم

```
import os
import platform

def system_info():
    print("--- System info ---")
    # درخواست نام سیستم عامل
    print(f"OS Type: {os.name}")
    print(f"Platform: {platform.system()} {platform.release()}")

    # پرسیدن شناسه از کرنل: "من که هستم؟"
    print(f"Current Process ID (PID): {os.getpid()}")

    # بررسی مسیر فعلی
    print(f"Current Directory: {os.getcwd()}")

if __name__ == "__main__":
    system_info()
```


کالبدشکافی کد

```
import os
import platform

def system_info():
    print("--- System info ---")
    # درخواست نام سیستم عامل
    print(f"OS Type: {os.name}")
    print(f"Platform: {platform.system()} {platform.release()}")

    # پرسیدن شناسه از کرنل: "من که هستم؟"
    print(f"Current Process ID (PID): {os.getpid()}")

    # بررسی مسیر فعلی
    print(f"Current Directory: {os.getcwd()}")

if __name__ == "__main__":
    system_info()
```

شناسایی سیستم: پرسش از کرنل
درباره نوع سیستم عامل.

بسیار مهم: دریافت شناسه
منحصر به فرد (PID) از جدول
پروسس های کرنل.

برنامه های سیستمی
با این شناسه ها
مدیریت می شوند.

موقعیتیابی: دریافت آدرس
دایرکتوری فعلی از فایل سیستم.

موقعیتیابی: دریافت آدرس
دایرکتوری فعلی از فایل سیستم.

شناسه پروسس (PID) چیست؟

هر برنامه‌ای که اجرا می‌شود، از نظر کرنل فقط یک عدد است.



وقتی روی آیکون برنامه کلیک می‌کنید، کرنل یک پروسس جدید می‌سازد و یک شماره (PID) به آن اختصاص می‌دهد. برای بستن یا مدیریت برنامه، فقط با این شماره سر و کار داریم.

مأموریت هفته آینده

Setup

محیط WSL و Ubuntu را نصب کنید.
دستور 'uname -a' را اجرا و اسکرینشات بگیرید.

Research

تحقیق کنید: تفاوت User Mode و Kernel Mode چیست؟
اگر در هر کدام کرش رخ دهد چه می‌شود؟

Coding

کدی بنویسید که با استفاده از 'os.environ' تمام متغیرهای محیطی سیستم را چاپ کند.

سیاست ارزشیابی: گُذِ کارآ

این کلاس تئوری محض نیست. خروجی شما باید کار کند.

پروژه‌ها شامل ابزارهای واقعی: اسکنر پورت، تحلیل بدافزار.

انعطاف‌پذیری: اجرا روی ویندوز یا لینوکس.

امتیاز ویژه: استفاده از Docker.

به دنیای زیر کاپوت خوش آمدید

منابع و مطالعه بیشتر

docs.python.org/3/library/os.html

learn.microsoft.com/windows/wsl

هفته بعد: مدیریت فایل‌های سطح پایین و WinAPI